

LAPORAN PRAKTIKUM 2
PEMROGRAMAN BERORIENTASI OBJEK
KONSEP DASAR PEMROGRAMAN BERORIENTASI OBJEK



Disusun Oleh:

Nama	: Mandanta Guru Singa
NIM	: 124140147
Mata Kuliah	: Pemrograman Berorientasi Objek
Dosen Pengampu	: Rahman Indra Kesuma, S. Kom., M. Cs.
Kelas	: RB

PROGRAM STUDI TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI SUMATERA
2026

1. Tujuan Praktikum

- Menjelaskan pengertian pemrograman berorientasi objek.
- Menjelaskan karakteristik dasar OOP.
- Memahami konsep class dan object.
- Memahami atribut, method, field, dan constructor.
- Menggunakan modifier sederhana pada class, atribut, dan method.
- Menggunakan keyword this pada program Java.
- Membuat program berbasis objek dengan contoh yang dekat dengan lingkungan ITERA.

2. Alat dan Bahan

- Laptop
- VS Code
- JDK versi 25
- Modul Praktikum PBO 2

3. Langkah Percobaan

3.1. Percobaan 1

- Membuat file Percobaan1.java.
- Mendefinisikan class Mahasiswa dengan atribut nama, nim, dan prodi.
- Melakukan instansiasi objek mhs menggunakan kata kunci new.
- Mengisi nilai atribut dan menampilkannya ke layar menggunakan System.out.println().

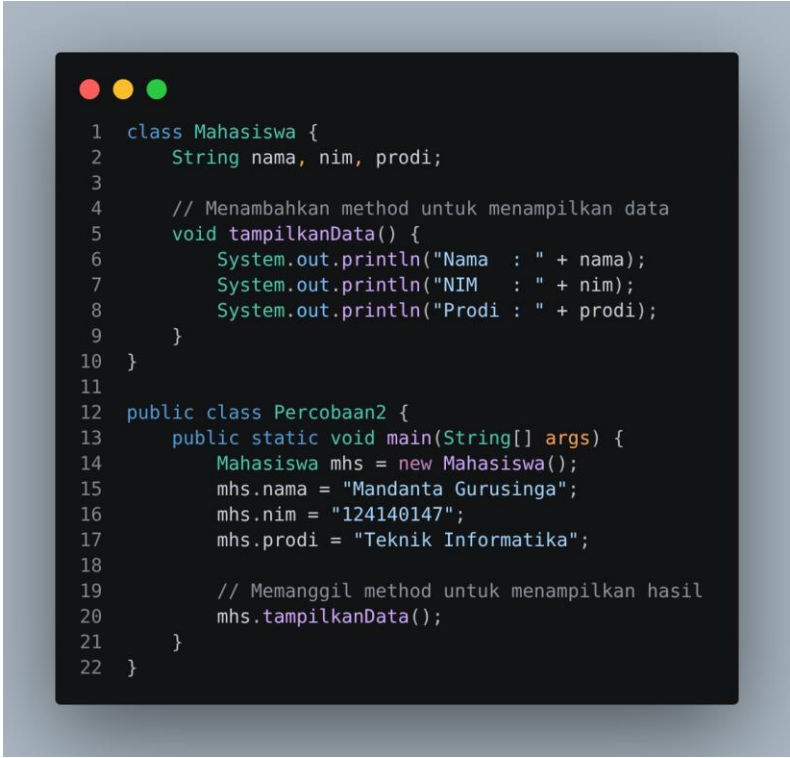


```
1  class Mahasiswa {
2      String nama;
3      String nim;
4      String prodi;
5  }
6
7  public class Percobaan1 {
8      public static void main(String[] args) {
9          Mahasiswa mhs = new Mahasiswa();
10
11          mhs.nama = "Mandanta Gurusinga";
12          mhs.nim = "124140147";
13          mhs.prodi = "Teknik Informatika";
14
15          System.out.println("Nama : " + mhs.nama);
16          System.out.println("NIM : " + mhs.nim);
17          System.out.println("Prodi : " + mhs.prodi);
18      }
19  }
```

3.2. Percobaan 2

- Memodifikasi class Mahasiswa dari percobaan sebelumnya.

- Menambahkan method bernama `tampilkanData()` yang berisi instruksi cetak atribut.
- Di dalam main, memanggil method tersebut menggunakan perintah `mhs.tampilkanData()`.



```

1  class Mahasiswa {
2      String nama, nim, prodi;
3
4      // Menambahkan method untuk menampilkan data
5      void tampilkanData() {
6          System.out.println("Nama : " + nama);
7          System.out.println("NIM : " + nim);
8          System.out.println("Prodi : " + prodi);
9      }
10 }
11
12 public class Percobaan2 {
13     public static void main(String[] args) {
14         Mahasiswa mhs = new Mahasiswa();
15         mhs.nama = "Mandanta Gurusinga";
16         mhs.nim = "124140147";
17         mhs.prodi = "Teknik Informatika";
18
19         // Memanggil method untuk menampilkan hasil
20         mhs.tampilkanData();
21     }
22 }

```

3.3. Percobaan 3

- Membuat class Dosen dengan atribut nama, nidn, dan prodi.
- Menambahkan method `tampilkanData()` di dalam class Dosen.
- Melakukan instansiasi dua object (`dsn1` dan `dsn2`) di dalam method main.
- Mengisi data masing-masing object dan memanggil method `tampilkanData()` untuk mencetak hasilnya.

```

1 class Dosen {
2     String nama, nidn, prodi;
3
4     // Method untuk menampilkan data dosen
5     void tampilkanData() {
6         System.out.println("Nama : " + nama);
7         System.out.println("NIDN : " + nidn);
8         System.out.println("Prodi : " + prodi);
9     }
10 }
11
12 public class Percobaan3 {
13     public static void main(String[] args) {
14         // Membuat object dosen pertama
15         Dosen dsn1 = new Dosen();
16         dsn1.nama = "Aidil Afriansyah";
17         dsn1.nidn = "19870101";
18         dsn1.prodi = "Informatika";
19
20         // Membuat object dosen kedua
21         Dosen dsn2 = new Dosen();
22         dsn2.nama = "Ibu Miranti";
23         dsn2.nidn = "19920202";
24         dsn2.prodi = "Informatika";
25
26         // Menampilkan data kedua dosen
27         System.out.println("Data Dosen 1:");
28         dsn1.tampilkanData();
29
30         System.out.println("Data Dosen 2:");
31         dsn2.tampilkanData();
32     }
33 }

```

3.4. Percobaan 4

- Membuat class MataKuliah dengan atribut namaMk dan sks.
- Menambahkan **Constructor** dengan parameter untuk menginisialisasi nilai atribut.
- Menggunakan keyword *this* untuk membedakan atribut *class* dengan parameter *constructor*.
- Melakukan instansiasi dua objek mata kuliah dengan memberikan argumen langsung pada tanda kurung *new MataKuliah(...)*.

```

1 class MataKuliah {
2     String namaMk;
3     int sks;
4
5     // Constructor untuk mengisi data secara otomatis saat objek dibuat
6     MataKuliah(String namaMk, int sks) {
7         this.namaMk = namaMk; // Menggunakan keyword this untuk merujuk ke atribut class
8         this.sks = sks;
9     }
10
11     // Method untuk menampilkan data
12     void tampilkanData() {
13         System.out.println("Mata Kuliah : " + namaMk);
14         System.out.println("SKS : " + sks);
15     }
16 }
17
18 public class Percobaan4 {
19     public static void main(String[] args) {
20         // Membuat object sekaligus mengisi data melalui constructor
21         MataKuliah mk1 = new MataKuliah("Pemrograman Berorientasi Objek", 3);
22         MataKuliah mk2 = new MataKuliah("Struktur Data", 3);
23
24         System.out.println("Daftar Mata Kuliah:");
25         mk1.tampilkanData();
26         mk2.tampilkanData();
27     }
28 }

```

3.5. Percobaan 5

- Membuat class Mahasiswa yang memiliki atribut lengkap (nama, nim, prodi, semester).

- Mengimplementasikan **Constructor** agar pengisian data lebih efisien saat instansiasi.
- Membuat **Method** tampilkanData() untuk mencetak seluruh atribut ke layar.
- Melakukan instansiasi **3 objek** mahasiswa dengan data berbeda di dalam main.
- Menjalankan program untuk memastikan seluruh data tampil dengan benar.

```

1 class Mahasiswa {
2     String nama, nim, prodi;
3     int semester;
4
5     // Constructor untuk mengisi data secara instan
6     Mahasiswa(String nama, String nim, String prodi, int semester) {
7         this.nama = nama;
8         this.nim = nim;
9         this.prodi = prodi;
10        this.semester = semester;
11    }
12
13    // Method untuk menampilkan data secara rapi
14    void tampilkanData() {
15        System.out.println("Nama      : " + nama);
16        System.out.println("NIM       : " + nim);
17        System.out.println("Prodi    : " + prodi);
18        System.out.println("Semester : " + semester);
19        System.out.println("-----");
20    }
21 }
22
23 public class Percobaan5 {
24     public static void main(String[] args) {
25         // Membuat minimal 3 objek mahasiswa ITERA
26         Mahasiswa mhs1 = new Mahasiswa("Mandanta Gurusinga", "124140147", "Informatika", 2);
27         Mahasiswa mhs2 = new Mahasiswa("Gaihan Mahendra", "124140201", "Informatika", 4);
28         Mahasiswa mhs3 = new Mahasiswa("Baginda Parulian Siregar", "124140135", "Sains Data", 2);
29
30         System.out.println("=== DATA MAHASISWA ITERA ===");
31         mhs1.tampilkanData();
32         mhs2.tampilkanData();
33         mhs3.tampilkanData();
34     }
35 }

```

4. Source Code

<https://github.com/DanamiStartrail/Tugas-2-Konsep-Dasar-PBO>

5. Hasil Output Program

5.1.Percobaan 1

```

$java Percobaan1
Nama : Mandanta Gurusinga
NIM  : 124140147
Prodi : Teknik Informatika

```

5.2.Percobaan 2

```

$java Percobaan2
Nama : Mandanta Gurusinga
NIM  : 124140147
Prodi : Teknik Informatika

```

5.3.Percobaan 3

```

$java Percobaan3
Prodi : Informatika
Data Dosen 2:
Nama  : Ibu Miranti
NIDN  : 19920202
Prodi : Informatika

```

5.4.Percobaan 4

```

Daftar Mata Kuliah:
Mata Kuliah : Pemrograman Berorientasi Objek
SKS          : 3
Mata Kuliah : Algoritma Struktur Data
SKS          : 3

```

5.5.Percobaan 5

```

$java Percobaan5
=== DATA MAHASISWA ITERA ===
Nama      : Mandanta Gurusinga
NIM       : 124140147
Prodi     : Informatika
Semester  : 2
-----
Nama      : Gathan Mahendra
NIM       : 124140201
Prodi     : Informatika
Semester  : 4
-----
Nama      : Baginda Parulian Siregar
NIM       : 124140135
Prodi     : Sains Data
Semester  : 2
-----

```

6. Analisis Program

6.1.Percobaan 1

- Class: Mahasiswa (sebagai cetak biru) dan Percobaan1 (kelas utama).
- Atribut: nama, nim, dan prodi bertipe String.
- Method: main sebagai titik awal eksekusi.
- Object: mhs sebagai hasil instansiasi dari class Mahasiswa.
- Hasil: Data atribut berhasil ditampilkan secara terstruktur ke terminal.

6.2.Percobaan 2

- Class: Mahasiswa dan Percobaan2.
- Atribut: nama, nim, dan prodi bertipe String.
- Method: Menambahkan tampilanData() sebagai perilaku (*behavior*) objek untuk mencetak informasi diri.
- Object: mhs sebagai hasil instansiasi.
- Hasil: Program lebih terstruktur karena logika tampilan dipisahkan dari logika utama (main).

6.3.Percobaan 3

- Class: Dosen (sebagai cetak biru) dan Percobaan3 (kelas utama).
- Atribut: nama, nidn, dan prodi bertipe String.
- Method: tampilanData() digunakan untuk menampilkan informasi spesifik dari tiap object dosen.
- Object: dsn1 dan dsn2 sebagai hasil instansiasi dari class Dosen.
- Hasil: Program berhasil mengelola dua data objek yang berbeda menggunakan satu *class* yang sama.

6.4.Percobaan 4

- Class: MataKuliah (cetak biru) dan Percobaan4 (kelas utama).
- Atribut: namaMk (String) dan sks (int).
- Method: tampilanData() untuk mencetak informasi objek.
- Constructor: Digunakan untuk menyederhanakan pengisian data atribut saat objek dibuat.
- Hasil: Objek berhasil dibuat dan datanya langsung terisi melalui *constructor*.

6.5.Percobaan 5

- Class: Mahasiswa sebagai cetak biru data civitas dan Percobaan5 sebagai kelas eksekusi.
- Atribut: Menggunakan tipe data String untuk teks dan int untuk angka (semester).
- Method: tampilanData() mempermudah pemanggilan informasi tanpa pengulangan kode System.out.println.
- Object: Berhasil membuat 3 entitas mahasiswa yang berdiri sendiri namun berbagi struktur yang sama.
- Hasil: Program berjalan sukses dan menampilkan data secara terstruktur sesuai format laporan yang diminta.

7. Kesimpulan

Berdasarkan praktikum BAB 2, saya menyimpulkan bahwa pemrograman berorientasi objek mempermudah pengelolaan data melalui konsep *class* dan *object*. Penggunaan *constructor* dan keyword *this* sangat efektif untuk inisialisasi data, sementara *method* membuat kode lebih modular dan mudah dikembangkan (*reusable*).